

T4NMTD: Transition-Centric Reinforcement Learning for Non-Markovian Task Decomposition - Technical Appendix

Anonymous submission

Evaluation

Experiment Setup

In this section, detailed experiment setup is provided. Table 1 presents the complete LTL_f formulas for all 8 NMTs.

Descriptions of the Domains

Domain 1 (Waterworld problem) The environment consists of a two-dimensional bounded arena containing balls of various colors. Each ball moves in a single direction at constant velocity and bounces elastically upon collision with the boundary walls. The agent, represented as a white ball, can accelerate or decelerate in any direction within the arena. We randomly generate maps for Waterworld with varying initial positions and velocities for all balls. Each map is constrained within a 30×30 boundary and contains 3 balls of each color, as illustrated in Figure 1. The agent’s objective is to touch different colored balls following specific sequential orders.

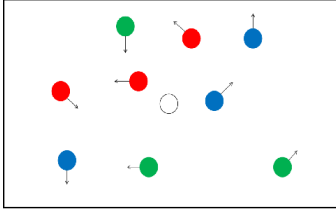


Figure 1: An instance of the Waterworld problem

Domain 2 (Halfcheetah problem) Referring to the benchmark in (Wawrzyński 2009), HalfCheetah designs a two-dimensional robot comprising 9 links and 8 joints connecting them (including two paws). The cheetah’s torso and head are fixed, and torque can only be applied to the other 6 joints over the front and back thighs (which connect to the torso), the shins (which connect to the thighs), and the feet (which connect to the shins). At each time step, the agent chooses how much force to apply to each joint in order to make the robot move forward or backward (from h_1 to h_5). The initial position of the cheetah (point h_3) and other milestone points (h_1, h_2, h_4, h_5) in HalfCheetah are shown in Figure 2.

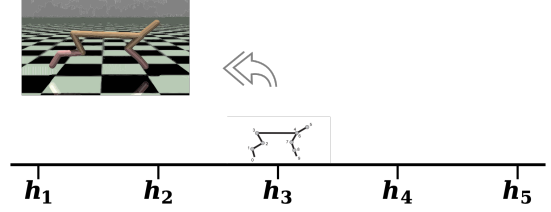


Figure 2: Depiction of the Halfcheetah problem

Domain 3 (Racecar problem) Racecar is a classical control domain originating from the International Planning Competition 2023 (Probabilistic and Reinforcement Learning Track)¹. In this domain, a race car must reach multiple targets (g_1, g_2, g_3, g_4) from an initial position in a specific order. The circular race track is bounded by lines representing the track limits, and hitting one of these boundaries results in failure, terminating the episode. The race car is modeled as a kinematic second-order system, while the track’s shape is defined by a series of points connected by lines. The problem setup is illustrated in Figure 3. As shown in the figure, both the agent and goal start at their designated initial positions, with the agent’s initial velocity set to zero.

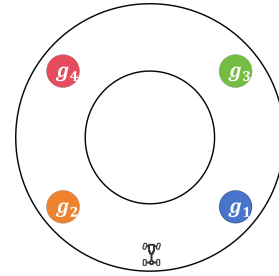


Figure 3: Depiction of the Racecar problem

Domain 4 (Frozenlake problem) Inspired by the frozen lake environment in Gym (Brockman et al. 2016), we designed the discrete Frozenlake problem. This domain involves navigating across a frozen lake from the start to the

¹<https://ataitler.github.io/IPPC2023/>

Environment	NMT Description	#states/#trans
Waterworld (continuous)	Task1: (r strict-then b strict-then g) and (B strict-then W strict-then G) $(\neg r \wedge \neg b \wedge \neg g)U(r \wedge \neg b \wedge \neg g \wedge \bigcirc((\neg r \wedge \neg b \wedge \neg g)U(b \wedge \neg r \wedge \neg g \wedge \bigcirc((\neg r \wedge \neg b \wedge \neg g)Ug))))$ $\wedge(\neg B \wedge \neg W \wedge \neg G)U(B \wedge \neg W \wedge \neg G \wedge \bigcirc((\neg B \wedge \neg W \wedge \neg G)U(W \wedge \neg B \wedge \neg G \wedge \bigcirc((\neg B \wedge \neg W \wedge \neg G)UG))))$	17/65
	Task2: (r strict-then b strict-then g strict-then p) and (B strict-then W strict-then G strict-then Y) $(\neg r \wedge \neg b \wedge \neg g \wedge \neg p)U(r \wedge \neg b \wedge \neg g \wedge \neg p \wedge \bigcirc((\neg r \wedge \neg b \wedge \neg g \wedge \neg p)U(b \wedge \neg r \wedge \neg g \wedge \neg p \wedge \bigcirc((\neg r \wedge \neg b \wedge \neg g \wedge \neg p)U(g \wedge \neg r \wedge \neg b \wedge \neg p \wedge \bigcirc((\neg r \wedge \neg b \wedge \neg g \wedge \neg p)U(p \wedge \neg r \wedge \neg b \wedge \neg g)))))))$ $\wedge(\neg B \wedge \neg W \wedge \neg G \wedge \neg Y)U(B \wedge \neg W \wedge \neg G \wedge \neg Y \wedge \bigcirc((\neg B \wedge \neg W \wedge \neg G \wedge \neg Y)U(W \wedge \neg B \wedge \neg G \wedge \neg Y \wedge \bigcirc((\neg B \wedge \neg W \wedge \neg G \wedge \neg Y)U(G \wedge \neg B \wedge \neg W \wedge \neg Y \wedge \bigcirc((\neg B \wedge \neg W \wedge \neg G \wedge \neg Y)U(Y \wedge \neg B \wedge \neg W \wedge \neg G)))))))$	26/106
Halfcheetah (continuous)	Task3: h_4 strict-then h_3 strict-then h_2 $(\neg h_2 \wedge \neg h_4)U(h_4 \wedge \bigcirc((\neg h_3 \wedge \neg h_5)U(h_3 \wedge \bigcirc((\neg h_2 \wedge h_4)Uh_2))))$	5/11
	Task4: (h_4 strict-then h_3 strict-then h_2) or (h_2 strict-then h_3 strict-then h_4) $(\neg h_2 \wedge \neg h_4)U(h_4 \wedge \bigcirc((\neg h_3 \wedge \neg h_5)U(h_3 \wedge \bigcirc((\neg h_2 \wedge \neg h_4)Uh_2))))$ $\vee(\neg h_2 \wedge \neg h_4)U(h_2 \wedge \bigcirc((\neg h_1 \wedge \neg h_3)U(h_3 \wedge \bigcirc((\neg h_2 \wedge \neg h_4)Uh_4))))$	9/25
Racecar (continuous)	Task5: g_1 then g_2 then g_3 then g_4 $\Diamond(g_1 \wedge \bigcirc(g_2 \wedge \bigcirc(g_3 \wedge \bigcirc g_4)))$	5/16
	Task6: (g_1 then g_2 then g_3 then g_4) or (g_4 then g_3 then g_2 then g_1) $\Diamond(g_1 \wedge \bigcirc(g_2 \wedge \bigcirc(g_3 \wedge \bigcirc g_4))) \vee \Diamond(g_4 \wedge \bigcirc(g_3 \wedge \bigcirc(g_2 \wedge \bigcirc g_1)))$	18/69
Frozenlake (discrete)	Task7: guide one of $\{p_1, p_2, p_3\}$ to g and only take one at a time $(\Diamond(p_1 \wedge \bigcirc(p_1 U(p_1 \wedge g))) \vee \Diamond(p_2 \wedge \bigcirc(p_2 U(p_2 \wedge g))) \vee \Diamond(p_3 \wedge \bigcirc(p_3 U(p_3 \wedge g))))$ $\wedge \Box(\neg(p_1 \wedge p_2) \wedge \neg(p_1 \wedge p_3) \wedge \neg(p_2 \wedge p_3))$	6/26
	Task8: guide one of $\{p_1, p_2\}$ to g_1 and p_3 to g_2 and only take one at a time $((\Diamond(p_1 \wedge \bigcirc(p_1 U(p_1 \wedge g_1))) \vee \Diamond(p_2 \wedge \bigcirc(p_2 U(p_2 \wedge g_1)))) \wedge \Diamond(p_3 \wedge \bigcirc(p_3 U(p_3 \wedge g_2))))$ $\wedge \Box(\neg(p_1 \wedge p_2) \wedge \neg(p_1 \wedge p_3) \wedge \neg(p_2 \wedge p_3))$	11/47

Table 1: Descriptions of the NMTs for *Waterworld*, *Halfcheetah*, *Racecar*, and *Frozenlake*

goal without falling into holes, while accounting for the slippery nature of the surface, which may cause the agent to move in unintended directions. Frozenlake is set in an 8×8 grid world, where the agent must transport specific passengers to their designated locations while avoiding falling into icy holes. Figure 4 provides a concrete illustration of the Frozenlake problem.



Figure 4: Depiction of the frozenlake problem

Illustration Examples

Example 1 (LTL_f to DFA) We use the waterworld problem to illustrate how LTL_f formulas specify tasks and their corresponding DFA. As shown in Figure 1, the environment contains three types of colored balls. The objective is to touch these balls in a specific sequential order: first touch a red ball, then touch a green ball. This represents a typical non-Markovian task (NMT) that can be formalized using

the LTL_f formula φ below, where propositions r and g denote touching a red ball and a green ball, respectively. The formula φ specifies that the agent must initially touch a red ball, after which it cannot touch any red balls again and must subsequently touch exactly one green ball.

$$\varphi = (\neg r \wedge \neg g) U((r \wedge \neg g) \wedge \bigcirc((\neg r \wedge \neg g)U(g \wedge \neg r)))$$

Note that touching blue balls is irrelevant to the satisfaction of φ . The corresponding DFA is illustrated in Figure 5. In this automaton, q_2 represents an error state indicating task failure, while q_4 (depicted with double circles) represents the accepting state indicating successful task completion. The task terminates when either q_4 (success) or q_2 (failure) is reached.

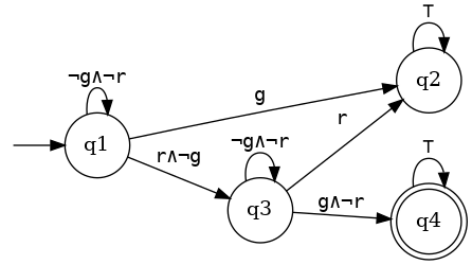


Figure 5: DFA of $\varphi = (\neg r \wedge \neg g) U((r \wedge \neg g) \wedge \bigcirc((\neg r \wedge \neg g)U(g \wedge \neg r)))$. $\neg g \wedge \neg r$, $g \wedge \neg r$, $r \wedge \neg g$ are logical representations of $\{g, r\}$, $\{g\}$, $\{r\}$ respectively. g represents $\{g, r\}$ or $\{g\}$, similar to r .

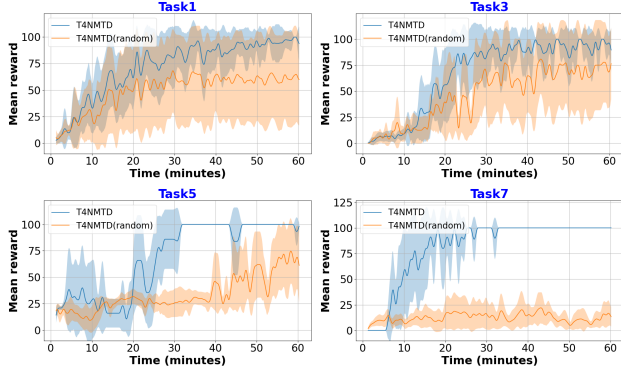


Figure 6: Comparison between proposition-based and random decomposition of NMTs

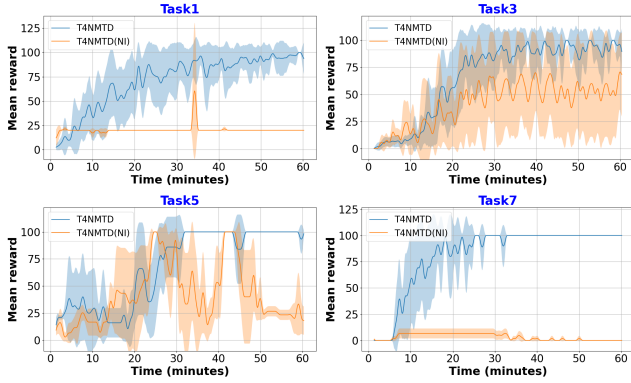


Figure 7: Comparison between T4NMTD with and without policy intervention

Example 2 (Task Decomposition) We use the DFA shown in Figure 5 to illustrate our task decomposition method. It can be observed that there are 4 sub-tasks with propositions r , g , $\neg r$, and $\neg g$. According to the defined decomposition mapping function $\mathcal{D}(\tau) = \mathcal{T}_l \mid \exists l : \tau = (q, l, q'), l \in l$, we can compute the transitions contained in each of these four sub-tasks. Specifically, $\mathcal{T}_r$ contains $(q_1, r \wedge \neg g, q_3)$, $\mathcal{T}_g$ contains $(q_3, g \wedge \neg r, q_4)$, $\mathcal{T}_{\neg r}$ contains $(q_3, g \wedge \neg r, q_4)$, and $\mathcal{T}_{\neg g}$ contains $(q_1, r \wedge \neg g, q_3)$.

Experimental Results

The second and third experiments in the main paper are conducted to demonstrate the effectiveness of the task decomposition mechanism and the policy intervention mechanism respectively. Figures 6 and 7 provide supplementary comparisons for the remaining tasks from these two experiments. The results are basically consistent with the tasks presented in the main paper.

Correctness and Optimality

In this section, we discuss the optimality and correctness of policies obtained by T4NMTD, and provide relevant theoretical proofs.

Definition 1 Given an NMDP $NM = \langle S, A, R = \langle \varphi, r \rangle, P, \gamma, s_0 \rangle$ and a DFA $\mathcal{A} = \langle Q, \Sigma, \delta, q_0, F \rangle$ with respect to the reward formula φ , NM is equivalent to the product of NM and $\mathcal{A}$, i.e., MDP $M^\otimes = \langle S^\otimes, A, R^\otimes, P^\otimes, \gamma, s_0^\otimes \rangle$, if there exist two functions $f_1 : S \times Q \rightarrow S^\otimes$ and $f_2 : S^\otimes \rightarrow S$ such that

1. $\forall s \in S : f_2(f_1(s)) = s$.
2. $\forall s_1, s_2 \in S, \forall s_1^\otimes \in S^\otimes$ and $\forall a \in A$: if $f_2(s_1^\otimes) = s_1$ and $P(s_2 \mid s_1, a) > 0$, there exists a unique $s_2^\otimes \in S^\otimes$ such that $f_2(s_2^\otimes) = s_2$ and $P^\otimes(s_2^\otimes \mid s_1^\otimes, a) = P(s_2 \mid s_1, a)$.
3. For any feasible trajectory $s_0 a_0 s_1 a_1 \dots s_n$ of NM and $s_0^\otimes a_0 s_1^\otimes a_1 \dots s_n^\otimes$ of $M^\otimes$, such that $f_1(s_0) = s_0^\otimes$ and $s_i = f_2(s_i^\otimes)$ for $0 \leq i \leq n$, we have $R(s_0 \dots s_{i+1}, a_i) = R^\otimes(s_i^\otimes, a_i, s_{i+1}^\otimes)$ for $0 \leq i < n$.

Theorem 1 An NMDP $NM = \langle S, A, R = \langle \varphi, r \rangle, P, \gamma, s_0 \rangle$ is equivalent to the product MDP $M^\otimes = \langle S^\otimes, A, R^\otimes, P^\otimes, \gamma, s_0^\otimes \rangle$, which is the product of NM and the DFA $\mathcal{A} = \langle Q, \Sigma, \delta, q_0, F \rangle$ with respect to the reward formula φ .

Proof. Recall that every $s^\otimes \in S^\otimes$ has the form $\langle s, q \rangle$. The reward function of NM is defined in terms of a pair $R = \langle \varphi, r \rangle$, where the agent get reward r only when the LTL $_f$ formula is satisfied, i.e.

$$R(s_0 \dots s_{n+1}, a_n) = \begin{cases} r, & \varphi \text{ is satisfied} \\ 0, & \text{otherwise} \end{cases}$$

Besides that, the transition probability in $M^\otimes$ is formalized below:

$$P^\otimes(s_{t+1}^\otimes \mid s_t^\otimes, a_t) = \begin{cases} P(s_{t+1} \mid s_t, a_t), & s_t^\otimes = \langle s_t, q_t \rangle, s_{t+1}^\otimes = \langle s_{t+1}, q_{t+1} \rangle, \\ & \exists l \in \Sigma : \langle q_t, l, q_{t+1} \rangle \in \delta, s_t \text{ satisfies } l \\ 0, & \text{otherwise} \end{cases} \quad (1)$$

The reward function in $M^\otimes$ is defined as

$$R^\otimes(s_t^\otimes, a_t, s_{t+1}^\otimes) = \begin{cases} r, & s_{t+1}^\otimes = \langle s_{t+1}, q_{t+1} \rangle, q_{t+1} \in F \\ 0, & \text{otherwise} \end{cases} \quad (2)$$

First we define $f_1(s) = \langle s, q_0 \rangle$ and $f_2(s^\otimes) = f_2(\langle s, q \rangle) = s$. Then condition 1 is easily verifiable by inspection. For condition 2, let $s_2^\otimes = \langle s_2, \delta(q_1, s_1) \rangle$, then the first condition of Equation 1 can be satisfied, we have $P^\otimes(s_2^\otimes \mid s_1^\otimes, a) = P(s_2 \mid s_1, a)$ and $f_2(s_2^\otimes) = s_2$. The above proves the existence of $s_2^\otimes$, in the following, we demonstrate the uniqueness. Assume that $s_2^\otimes = \langle s^+, \delta(q_1, s_1) \rangle$ where $s^+ \neq s_2 \in S$ and $s^+ \in S$, it cannot hold that $\forall s_1, s_2 \in S : f_2(s_1^\otimes) = s_1$ and $f_2(s_2^\otimes) = s_2$. Assume that $s_2^\otimes = \langle s_2, q^+ \rangle$ where $q^+ \neq \delta(q_1, s_1)$, then $P^\otimes(s_2^\otimes \mid s_1^\otimes, a) = 0 \neq P(s_2 \mid s_1, a)$ based on Equation 1. Hence we can conclude the uniqueness.

For condition 3, since $f_1(s_0) = s_0^\otimes$ and $s_i = f_2(s_i^\otimes)$ for $0 \leq i \leq n$, we have $s_{i+1}^\otimes = \langle s_{i+1}, \delta(q_i, s_i) \rangle$ according to condition 2. Then a DFA trace $q_0 \dots q_n$ can be induced by

state trace $s_0 \dots s_n$. We can discuss condition 3 in two cases. When φ is satisfied after the trajectory $s_0 a_0 s_1 a_1 \dots s_n$, reward r is received in NM based on $R = \langle \phi, r \rangle$. Namely, we have the DFA trace $q_0, q_1, \dots, q_{n+1}$ such that $q_{n+1} \in F$. Hence the reward also r in $M^\otimes$, according to Equation 2. When the task is not completed, the rewards of NM and $M^\otimes$ are both zero.

Next, we discuss the equivalence of the optimal policies between $M^\otimes$ and NM . Let $\pi^\otimes(a \mid s^\otimes)$ be a policy of $M^\otimes$, which determines the probability of an action $a \in A$ at each state $s^\otimes \in S^\otimes$. It is easy to define the equivalent policy of NM , i.e., $\pi(a \mid s_0 \dots s_n)$. Given any feasible trajectory $s_0 a_0 s_1 a_1 \dots s_n$ of NM and DFA trace $q_0 \dots q_{n+1}$ induced by state trace $s_0 \dots s_n$, we have $\forall a \in A : \pi(a \mid s_0 \dots s_n) = \pi^\otimes(a \mid \langle s_n, q_n \rangle)$.

Theorem 2 *Given a product MDP $M^\otimes$, let $\pi^{\otimes'}$ be the optimal policy of $M^\otimes$. Then, the policy π' that is equivalent to $\pi^{\otimes'}$ is optimal for NM .*

Proof. We prove the following lemma firstly.

Lemma 1 *For any feasible trajectory $s_0 a_0 s_1 a_1 \dots s_n$ of NM and $s_0^\otimes a_0 s_1^\otimes a_1 \dots s_n^\otimes$ of $M^\otimes$, such that $f_1(s_0) = s_0^\otimes$ and $s_i = f_2(s_i^\otimes)$ for $0 \leq i \leq n$, we have*

$$\begin{aligned} Pr_{\pi'}(s_0 a_0 s_1 a_1 \dots s_n) &= Pr_{\pi^{\otimes'}}(s_0^\otimes a_0 s_1^\otimes a_1 \dots s_n^\otimes) \\ \mathbb{E} \left[\sum_{k=0}^{n-1} \gamma^k R^\otimes(s_k^\otimes, a_k, s_{k+1}^\otimes) \right] &= \\ \mathbb{E} \left[\sum_{k=0}^{n-1} \gamma^k R(s_0 \dots s_{k+1}, a_k) \right] \end{aligned}$$

where $Pr_\pi(\cdot)$ denotes the probability of generating a trajectory under policy π .

Proof. $Pr_{\pi'}$ and $Pr_{\pi^{\otimes'}}$ can be transformed in the following form:

$$\begin{aligned} Pr_{\pi'}(s_0 a_0 s_1 a_1 \dots s_n) &= \pi(a_0 \mid s_0) P(s_1 \mid s_0, a_0) \\ &\times \pi(a_1 \mid s_0 s_1) P(s_2 \mid s_1, a_1) \dots \\ &\times \pi(a_{n-1} \mid s_n \dots s_{n-1}) P(s_n \mid s_{n-1}, a_{n-1}) \\ Pr_{\pi^{\otimes'}}(s_0^\otimes a_0 s_1^\otimes a_1 \dots s_n^\otimes) &= \pi^{\otimes'}(a_0 \mid s_0^\otimes) P(s_1^\otimes \mid s_0^\otimes, a_0) \\ &\times \pi^{\otimes'}(a_1 \mid s_1^\otimes) P(s_2^\otimes \mid s_1^\otimes, a_1) \dots \\ &\times \pi^{\otimes'}(a_{n-1} \mid s_{n-1}^\otimes) P(s_n^\otimes \mid s_{n-1}^\otimes, a_{n-1}) \end{aligned}$$

It is easy to obtain that $P^\otimes(s_{i+1}^\otimes \mid s_i^\otimes, a) = P(s_{i+1} \mid s_i, a)$ and $R(s_0 \dots s_{i+1}, a_i) = R^\otimes(s_i^\otimes, a_i, s_{i+1}^\otimes)$ for $0 \leq i < n$ according to Conditions 2 and 3. Then Lemma 1 can be easily verifiable by inspection.

In the following, we demonstrate Theorem 2 by contradiction. Since $\pi^{\otimes'}$ is the optimal policy of $M^\otimes$, at each state $s^\otimes \in S^\otimes$ we have

$$\pi^{\otimes'} = \operatorname{argmax}_{\pi^{\otimes'} \in D^\otimes} \mathbb{E} \left[\sum_{k=0}^{n-1} \gamma^k R^\otimes(s_k^\otimes, a_k, s_{k+1}^\otimes) \right] \quad (3)$$

where $D^\otimes$ denotes the set of policies over the state space $S^\otimes$, and $s_k^\otimes, a_k, s_{k+1}^\otimes, a_{k+1}, \dots$ is a generic path generated by $M^\otimes$ under policy $\pi^\otimes$. Similarly, the optimal policy of NM maximizes the expected accumulated rewards. Assume that there exists a policy $\pi^+ \neq \pi'$ and a possible trace $s_0 a_0 s_1 a_1 \dots s_n$ such that

$$\begin{aligned} \mathbb{E}_{s_n \sim \pi^+} \left[\sum_{k=n}^{T-1} \gamma^k R(s_0 \dots s_{k+1}, a_k) \right] \\ > \mathbb{E}_{s_n \sim \pi'} \left[\sum_{k=n}^{T-1} \gamma^k R(s_0 \dots s_{k+1}, a_k) \right] \end{aligned}$$

According to Lemma 1, for the trajectory $s_0^\otimes a_0 s_1^\otimes a_1 \dots s_n^\otimes$ of $M^\otimes$, such that $f_1(s_0) = s_0^\otimes$ and $s_i = f_2(s_i^\otimes)$ for $0 \leq i \leq n$, we have

$$\begin{aligned} \mathbb{E}_{s_n^\otimes \sim \pi^{\otimes+}} \left[\sum_{k=n}^{T-1} \gamma^k R(s_k^\otimes, a_k, s_{k+1}^\otimes) \right] \\ > \mathbb{E}_{s_n^\otimes \sim \pi^{\otimes'}} \left[\sum_{k=n}^{T-1} \gamma^k R(s_k^\otimes, a_k, s_{k+1}^\otimes) \right] \end{aligned}$$

This is in contrast with optimality of $\pi^{\otimes*}$ and then Theorem 2 can be proved.

Theorem 3 *Given a product MDP $M^\otimes = \langle S^\otimes, A, R^\otimes, P^\otimes, \gamma, s_0^\otimes \rangle$ converted by an NMDP and a DFA with respect to a reward formula ϕ . Then the optimal policy $\pi^{\otimes*}$ of $M^\otimes$ minimizes the expected steps to satisfying ϕ when $\gamma < 1$ or maximizes the probability of satisfying φ when $\gamma = 1$.*

Proof. Equation 3 has the form below at initial state $s_0^\otimes$

$$\begin{aligned} \pi^{\otimes*}(s_0^\otimes) &= \operatorname{argmax}_{\pi^{\otimes*} \in D^\otimes} \mathbb{E} \left[\sum_{k=0}^{T-1} \gamma^k R^\otimes(s_k^\otimes, a_k, s_{k+1}^\otimes) \right] \\ &= \operatorname{argmax}_{\pi^{\otimes*} \in D^\otimes} \mathbb{E} [\gamma^L R^\otimes(s_L^\otimes, a_L, s_{L+1}^\otimes)] \\ &= \operatorname{argmax}_{\pi^{\otimes*} \in D^\otimes} P(s_0^\otimes) \gamma^L r \end{aligned} \quad (4)$$

where L is the steps to satisfying φ and $P(s_0^\otimes)$ indicates the probability of satisfying φ . Since reward is only given when ϕ is satisfied, when $\gamma < 1$, maximizes the expected accumulated rewards for $\pi^{\otimes*}$ is equivalent to minimizing the steps according to Equation 4. When $\gamma = 1$, then $\pi^{\otimes*}(s_0^\otimes) = P(s_0^\otimes)r$, hence $\pi^{\otimes*}$ maximizes the probability of satisfying ϕ .

Finally, we discuss whether T4NMTD can converge to the optimal strategy. Due to the strong generalizability, T4NMTD can set various algorithms as baselines, so the convergence depends on the specifics of the baseline algorithm. Here, we demonstrate this with T4NMTD and Q-learning as an example. Similar proofs can be referenced when other algorithms are used in T4NMTD as baselines.

Similarly, Q-learning has the following update format (referring to (Melo 2001))

$$\begin{aligned} Q'(s_i^\otimes, a_i) &= Q(s_i^\otimes, a_i) + \alpha_i(s_i^\otimes, a_i) \cdot \\ &\left[R(s_i^\otimes, a_i, s_{i+1}^\otimes) + \gamma \max_{b \in A} Q(s_{i+1}^\otimes, b) - Q(s_i^\otimes, a_i) \right] \end{aligned}$$

The key insight is that both update rules follow the same Bellman optimality principle. This leads to the following corollary:

Corollary 1 *Given the Q-learning update rule (following (Melo 2001)):*

$$Q_{t+1}(s_t, a_t) = Q_t(s_t, a_t) + \alpha_t(s_t, a_t) \cdot \left[R(s_t, a_t, s_{t+1}) + \gamma \max_{a' \in A} Q_t(s_{t+1}, a') - Q_t(s_t, a_t) \right]$$

Suppose that for a given state s_{t+1} , the Q-values $Q(s_{t+1}, a')$ have converged to their optimal values $Q^(s_{t+1}, a')$ for all $a' \in A$. Then, under the standard convergence conditions for Q-learning (appropriate learning rate schedule and infinite visitation), the Q-value $Q(s_t, a_t)$ will converge to its optimal value $Q^*(s_t, a_t)$.*

Theorem 4 *Given a product MDP $M^\otimes = \langle S^\otimes, A, R^\otimes, P^\otimes, \gamma, s_0^\otimes \rangle$ and the DFA $\mathcal{A} = \langle Q, \Sigma, \delta, q_0, F \rangle$, T4NMTD with tabular Q-Learning converges to an optimal policy as long as every possible state-action pair is visited infinitely often.*

Proof.

In T4NMTD, each modular Q-function Q_ι is updated by the rule

$$Q'_\iota(s_i^\otimes, a_i) = Q_\iota(s_i^\otimes, a_i) + \alpha_i(s_i^\otimes, a_i) \cdot \left[R(s_i^\otimes, a_i, s_{i+1}^\otimes) + \gamma \max_{b \in A} Q_{\iota'}(s_{i+1}^\otimes, b) - Q_\iota(s_i^\otimes, a_i) \right]$$

where $s_i^\otimes = \langle s_i, q_i \rangle$ and $\iota' \sim \mathcal{E}(Q_h, q_i)$ if $q_i \neq q_{i+1}$ and $q_{i+1} \notin F \cup E$ else $\iota' = \iota$.

It should be noted that the key difference between our approach and standard Q-learning lies in the computation of action values for the next state. In our method, this computation is not necessarily performed using the same Q-table, but rather depends on $\iota' \sim \mathcal{E}(Q_h, q_i)$ (policy intervention). In other words, when the action values within a module are computed solely by itself, the module will converge to the optimal solution for that subtask, similar to standard Q-learning. Therefore, when learning a transition (q_i, l, q_{i+1}) where $q_{i+1} \in F$ (i.e., reaching a final state), we have $\iota' = \iota$. This means that for this transition, $Q(s^\otimes = \langle s, q_i \rangle, \cdot)$ will converge to the optimal value since it follows the standard Q-learning update rule. By Corollary 1, once this Q-function has converged, the transitions that precede this transition will also converge to their optimal values. This convergence propagates backward through the sequence of transitions until the entire action-value function converges to the global optimum.

References

- Brockman, G.; Cheung, V.; Pettersson, L.; Schneider, J.; Schulman, J.; Tang, J.; and Zaremba, W. 2016. Openai gym. *arXiv preprint arXiv:1606.01540*.
- Melo, F. S. 2001. Convergence of Q-learning: A simple proof. *Institute Of Systems and Robotics, Tech. Rep.*, 1–4.

- Wawrzyński, P. 2009. A cat-like robot real-time learning to run. In *Adaptive and Natural Computing Algorithms: 9th International Conference, ICANNGA 2009, Kuopio, Finland, April 23-25, 2009, Revised Selected Papers 9*, 380–390. Springer.